

Gluwa CEX SmartAccounts Security Audit

: CEX SmartAccounts Contract Security Audit

March 13, 2026

Revision 1.1

ChainLight@Theori

Theori, Inc. ("We") is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

Table of Contents

Gluwa CEX SmartAccounts Security Audit	1
Table of Contents	2
Executive Summary	4
Audit Overview	5
Scope	5
Code Revision	6
Severity Categories	6
Status Categories	8
Finding Breakdown by Severity	9
Findings	10
Summary	10
#1 GLUWASA-001 MasterForwarder role wallet changes should use a two-step transfer mechanism	11
Revision History	14

Executive Summary

From Feb 11 to Feb 12, 2026, Theori's ChainLight team conducted a 2-day security audit of Gluwa's CEX SmartAccounts contracts. The assessment focused on the `CEX SmartAccounts` system, in which a factory contract can deploy `DepositForwarder` instances as clones at deterministic and predictable addresses using `CREATE2`. Each `DepositForwarder` is designed to receive ETH or ERC-20 tokens and forward them to the address returned by `destination()` in the `MasterForwarder` contract. The review covered the correctness of deterministic `CREATE2` address derivation, whether clone deployment through the factory could be influenced or manipulated by a malicious caller, whether unauthorized users could trigger `forward()`, and how privileged roles are managed within `MasterForwarder`, including the restrictions that only `operatorWallet()` may call `DepositForwarder.forward()` and only `liquidityManagerWallet` may update the destination address in `MasterForwarder`.

During the audit, one informational issue was identified in `MasterForwarder.sol`. Specifically, the `changeOperatorWallet()` and `changeLiquidityManagerWallet()` functions transferred privileged roles in a single step without requiring explicit acceptance by the new address, which could increase operational risk if an incorrect or unusable address were assigned. The issue was addressed during the audit by adopting a safer wallet transfer flow consistent with the two-step acceptance pattern used in OpenZeppelin's `Ownable2Step`.

Overall, the review confirmed that the factory's deployment flow correctly preserves deterministic deployment behavior, that clone deployment does not introduce meaningful manipulation opportunities for external callers, and that forwarding functionality is properly restricted to the designated privileged role. Following remediation of the informational issue, the reviewed contracts were found to operate consistently with their intended design.

Audit Overview

Scope

Name	Gluwa CEX SmartAccounts Security Audit
Target / Version	• Git Repository (<code>gluwa/CEX-SmartAccounts</code>): commit <code>8fedbd62788110409793037d09be954d9461a43c</code> (excluding <code>MultiCall3.sol</code>) • Note: An additional audit of commit <code>79a2f21d4f10271dc3d78a2aa0b787839ae4c0e8</code> was conducted on March 13, 2026.
Application Type	Smart contracts
Lang. / Platforms	Smart contracts [Solidity]

Code Revision

N/A

Severity Categories

Severity	Description
Critical	The attack cost is low (not requiring much time or effort to succeed in the actual attack), and the vulnerability causes a high-impact issue. (e.g., Effect on service availability, Attacker taking financial gain)
High	An attacker can succeed in an attack which clearly causes problems in the service's operation. Even when the attack cost is high, the severity of the issue is considered "high" if the impact of the attack is remarkably high.
Medium	An attacker may perform an unintended action in the service, and the action may impact service operation. However, there are some restrictions for the actual attack to succeed.
Low	An attacker can perform an unintended action in the service, but the action does not cause significant impact or the success rate of the attack is remarkably low.
Informational	Any informational findings that do not directly impact the user or the protocol.
Note	Neutral information about the target that is not directly related to the project's safety and security.

Status Categories

Status	Description
Reported	ChainLight reported the issue to the client.
WIP	The client is working on the patch.
Patched	The client fully resolved the issue by patching the root cause.
Mitigated	The client resolved the issue by reducing the risk to an acceptable level by introducing mitigations.
Acknowledged	The client acknowledged the potential risk, but they will resolve it later.
Won't Fix	The client acknowledged the potential risk, but they decided to accept the risk.

Finding Breakdown by Severity

Category	Count	Findings
Critical	0	N/A
High	0	N/A
Medium	0	N/A
Low	0	N/A
Informational	1	GLUWASA-001
Note	0	N/A

Findings

Summary

#	ID	Title	Severity	Status
1	GLUWASA-001	MasterForwarder role wallet changes should use a two-step transfer mechanism	Informational	Patched

#1 **GLUWASA-001** MasterForwarder role wallet changes should use a two-step transfer mechanism

ID	Summary	Severity
GLUWASA-001	In <code>MasterForwarder.sol</code> , the <code>changeOperatorWallet()</code> and <code>changeLiquidityManagerWallet()</code> functions immediately transfer privileged roles to the new address in a single step. Introducing a two-step acceptance flow, similar to <code>Ownable2Step</code> , would reduce the risk of accidentally assigning these roles to an incorrect or unusable address.	Informational

Description

In `MasterForwarder.sol`, both `changeOperatorWallet()` and `changeLiquidityManagerWallet()` update the role holder immediately once called by the current authorized wallet. While access control is properly enforced, the transfer is finalized in a single transaction without requiring any confirmation from the new address.

This creates operational risk. If the current role holder mistakenly provides the wrong address, or specifies an address that cannot actually manage the role as intended, the privileged role is transferred instantly and may become difficult to recover. Since these roles appear to control sensitive operational functionality, accidental misconfiguration could lead to administrative disruption or temporary loss of control over privileged actions.

A safer design is to follow a two-step transfer pattern, where the current role holder nominates a pending address and the new address must explicitly accept the role before the transfer is finalized.

Impact

Informational

Recommendation

Consider updating the wallet change flows in `MasterForwarder.sol` to use a two-step acceptance process similar to OpenZeppelin's `Ownable2Step`.

References

<https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/access/Ownable2Step.sol>

Remediation

Patched

commit: `c4fccec58d2f4a19bdad74c2339e4c298c5867d4`

Revision History

Version	Date	Description
1.0	March 10, 2026	Initial version
1.1	March 13, 2026	Added an additional audit for commit `79a2f21d4f10271dc3d78a2aa0b787839ae4c0e8`

Theori, Inc. ("We") is acting solely for the client and is not responsible to any other party. Deliverables are valid for and should be used solely in connection with the purpose for which they were prepared as set out in our engagement agreement. You should not refer to or use our name or advice for any other purpose. The information (where appropriate) has not been verified. No representation or warranty is given as to accuracy, completeness or correctness of information in the Deliverables, any document, or any other information made available. Deliverables are for the internal use of the client and may not be used or relied upon by any person or entity other than the client. Deliverables are confidential and are not to be provided, without our authorization (preferably written), to entities or representatives of entities (including employees) that are not the client, including affiliates or representatives of affiliates of the client.

